Performance Evaluation of Nav2 Controllers for Human-Aware Indoor Robot Navigation

Bhagyath Badduri
CPE 631-A Final Project
Stevens Institute of Technology
Hoboken, NJ, USA
bbadduri@stevens.edu

***Abstract*—Autonomous mobile robots operating in indoor human environments must safely reach a goal while avoiding static obstacles and moving pedestrians. This project evaluates robot navigation in a simulated cafe environment using ROS 2, Gazebo, RViz, and the Nav2 navigation stack [1], [7], [8]. A TurtleBot3 Burger robot is used as the mobile robot platform, and pedestrians are enabled to create a dynamic navigation scenario. The project compares two Nav2 controller configurations using the same environment and global planner: NavFn with MPPI and NavFn with DWB [2]–[4]. The performance is evaluated using qualitative observations from RViz and Gazebo, along with quantitative metrics such as total navigation time, robot path distance, minimum laser scan distance, and close encounters near pedestrians. The objective is to compare how the two controller configurations perform in terms of goal reaching, path following, obstacle avoidance, and safe navigation around humans.**

***Index Terms*—ROS 2, Nav2, TurtleBot3, NavFn, MPPI, DWB, mobile robot navigation, dynamic obstacles, human-aware navigation**

I. Introduction

Autonomous mobile robots are commonly used in indoor environments such as offices, hospitals, warehouses, and public spaces. In these environments, the robot must move safely while avoiding walls, furniture, and people. Navigation becomes more difficult when pedestrians are present because humans may cross the robot path, stop near the goal, or temporarily block the planned route. Therefore, the robot must continuously update its local environment information and generate safe motion commands.

This project focuses on indoor robot navigation in a pedestrian-enabled cafe simulation environment. The robot is required to move from a start location to a selected goal location while avoiding both static obstacles and moving pedestrians. The simulation is performed using ROS 2, Gazebo, RViz, and the Nav2 navigation stack. The TurtleBot3 Burger robot is used because it is suitable for differential-drive indoor navigation experiments.

The Nav2 stack includes the main components required for autonomous navigation [7]. The map server loads the cafe map, AMCL estimates the robot pose, the global planner generates a path to the goal, and the local controller generates velocity commands to follow the path while avoiding nearby obstacles. Costmaps are used to represent static and dynamic obstacles, inflated safety regions, and local navigation constraints around the robot [5]. In this project, NavFn was used as the global planner, while MPPI and DWB were tested as local controller configurations [2]–[4].

In this project, NavFn is used as the global planner. Two local controller configurations are tested and compared. The first configuration uses NavFn with the Model Predictive Path Integral controller. The second configuration uses NavFn with the Dynamic Window Approach controller. The main difference between the two test cases is the local controller, while the robot model, map, and pedestrian-enabled simulation remain the same.

A custom metrics logger is used to record the performance of each navigation run. The collected metrics include total navigation time, robot path distance, minimum laser scan distance, and the number of close encounters below a selected distance threshold. These metrics are used to compare the navigation behavior of NavFn with MPPI and NavFn with DWB under similar dynamic conditions.

II. Problem Formulation

The main problem addressed in this project is autonomous navigation in an indoor environment with moving pedestrians. The robot must reach a selected goal while avoiding static obstacles and humans. In a dynamic environment, the shortest path may not always be the safest path because pedestrians can move across the robot trajectory or stop near the goal. Therefore, the navigation system must balance goal reaching, path following, obstacle avoidance, and safe behavior around humans.

The project uses a TurtleBot3 Burger robot in a simulated cafe environment. The cafe map is already available, so the task is not mapping but navigation using localization, planning, and control. The robot uses odometry and laser scan data during navigation. The laser scan data helps the local costmap detect nearby obstacles and pedestrians, while AMCL is used to localize the robot on the known map.

The global planning component is kept the same for both experiments. NavFn is used to generate the global path from the robot's current pose to the goal pose. The local controller is changed between the two test cases. In the first test, MPPI is used as the controller. In the second test, DWB is used as the controller. This setup allows the effect of the local controller to be compared under the same map, robot model, pedestrian environment, and global planner.

The main assumptions for this project are:

- The cafe occupancy grid map is constructed first using teleoperation and then saved for navigation testing.
- The robot operates on a flat indoor surface.
- The TurtleBot3 Burger is treated as a differential-drive mobile robot.
- Pedestrians are considered dynamic obstacles in the simulation.
- The same start area, goal area, map, and pedestrian-enabled environment are used for comparison.
- NavFn is used as the global planner for both test configurations.
- MPPI and DWB are compared as local controllers.

The goal of the experiment is to evaluate which controller configuration provides better navigation behavior in the dynamic cafe environment. The comparison is based on whether the robot reaches the goal, how long it takes, how much distance it travels, how close it gets to pedestrians, and how smoothly it avoids moving obstacles.

III. Motion Planning and Navigation Algorithm Details

The navigation system in this project is implemented using the Nav2 stack in ROS 2 [7]. Nav2 provides the main modules required for autonomous mobile robot navigation, including map loading, localization, path planning, local control, costmap generation, and goal execution. In this project, the robot is tested in a known cafe map with moving pedestrians enabled in the simulation.

The navigation stack is divided into global planning and local control. The global planner is responsible for generating a path from the robot's current pose to the selected goal pose. The local controller is responsible for following that path while reacting to nearby obstacles and pedestrians. This separation allows the robot to have a planned route to the goal while still responding to changes in the local environment.

A. Nav2 Navigation Stack

The main Nav2 components used in this project are the map server, AMCL, planner server, controller server, costmaps, and BT navigator. The map server loads the cafe occupancy grid map. AMCL localizes the robot on the map using laser scan and odometry information. The planner server generates the global path to the goal. The controller server computes velocity commands for the robot. The BT navigator manages the navigation behavior when a goal is sent from RViz.

The global and local costmaps are important for obstacle avoidance. The global costmap represents the known map and obstacle information used for global planning. The local costmap is centered around the robot and updates during motion using sensor data. In this project, moving pedestrians are treated as dynamic obstacles through the robot's laser scan observations.

B. NavFn Global Planner

NavFn was used as the global planner for both controller configurations [2]. It generates a global path from the robot's current pose to the goal pose using the occupancy grid map. Keeping NavFn the same for both experiments makes the comparison fair because the global path planning method is not changed.

In both test cases, NavFn provides the reference path that the local controller follows. The difference between the two experiments comes from the local controller behavior. This means that any difference in navigation time, path distance, stopping behavior, or obstacle avoidance is mainly due to the selected local controller.

C. MPPI Local Controller

The first local controller configuration used the Model Predictive Path Integral controller [3]. MPPI is a sampling-based controller that evaluates possible robot motion commands over a short prediction horizon. It selects a velocity command based on the cost of predicted trajectories. The selected command should help the robot follow the global path, move toward the goal, and avoid nearby obstacles.

In this project, the MPPI controller is used with the NavFn global planner. The robot uses this configuration to navigate through the dynamic cafe environment while pedestrians are moving. The performance of this configuration is evaluated using the collected metrics and visual behavior observed in RViz and Gazebo.

D. DWB Local Controller

The second local controller configuration used the Dynamic Window Approach controller [4]. DWB evaluates possible linear and angular velocity commands within the robot's motion limits. Each candidate command is scored using different critics such as path following, goal distance, obstacle distance, and goal orientation. The command with the best score is selected and sent to the robot.

In this project, DWB is used with the same NavFn global planner and the same cafe environment. The purpose is to evaluate how DWB performs relative to MPPI in reaching the goal, avoiding pedestrians, and maintaining smooth motion.

E. Comparison Strategy

The comparison is performed by keeping the main simulation conditions the same and changing only the local controller. Both configurations use the same TurtleBot3 Burger model, cafe map, pedestrian-enabled simulation, and NavFn global planner. The first configuration is NavFn with MPPI, and the second configuration is NavFn with DWB.

The performance comparison is based on the following metrics:

- Total navigation time
- Robot path distance
- Minimum laser scan distance
- Number of close encounters below the selected threshold

- Qualitative goal-reaching and obstacle-avoidance behavior

This comparison helps evaluate how each local controller performs in a dynamic human environment. The results are used to identify which configuration provides more reliable and safer navigation for the simulated cafe scenario.

IV. Environment and GUI Interface

The project environment was set up using ROS 2 Jazzy, Gazebo, RViz, TurtleBot3 Burger, and the CPE631 Navigation ROS 2 package [1], [7], [8]. Gazebo was used to simulate the physical cafe environment, including the robot, static obstacles, and moving pedestrians. RViz was used to visualize the map, robot pose, global path, local costmap, global costmap, laser scan data, and navigation goal.

The cafe environment was selected because it represents an indoor public space where a mobile robot may need to navigate around furniture and people. The known cafe map was loaded using the Nav2 map server. After the map was loaded, the robot was localized using AMCL. The initial pose was assigned in RViz, and navigation goals were provided using the 2D Goal Pose tool.

A. Simulation Environment

The simulation contains a TurtleBot3 Burger robot inside a cafe layout. The cafe environment includes walls, tables, and walking pedestrians. The pedestrians create dynamic obstacles that can cross the robot's path during navigation. This makes the navigation problem more realistic compared to a static-only environment.

Gazebo was used to observe the physical movement of the robot and pedestrians. RViz was used to observe the navigation stack output, including the robot location on the map, planned path, and costmap updates. Both tools were used together to confirm whether the robot was reaching the goal correctly and avoiding obstacles.

B. RViz Visualization

RViz was used as the main graphical interface for navigation testing. The fixed frame was set to `map`. The map display was used to verify that the occupancy grid was loaded correctly. The robot model, laser scan, global path, local costmap, and global costmap were enabled to monitor the navigation behavior.

The 2D Pose Estimate tool was used to provide the robot's initial pose when required. The 2D Goal Pose tool was used to send navigation goals to the robot. After assigning a goal, RViz displayed the global path and the robot motion response. This helped verify whether the robot was following the planned path and reacting to pedestrians.

C. Static and Dynamic Navigation Setup

The project was tested in both static and dynamic navigation conditions. In the static setup, pedestrians were disabled and the robot navigated only around fixed obstacles. This was useful for verifying that the map, localization, planner, and controller were working correctly.

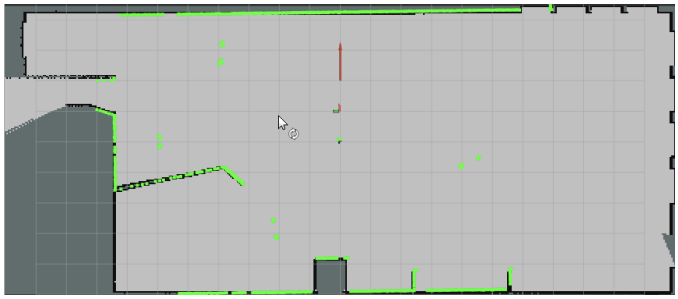

Fig. 1. RViz visualization of the cafe map, robot pose, laser scan, and navigation interface used during testing.

```
bhagyath@bbr:~/cpe631_ws$ cd ~/cpe631_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
export TURTLEBOT3_MODEL=burger
export RMW_FASTRTPS_USE_SHM=0

ros2 launch cpe631_ros2 cafe.launch.py \
  navigation:=true \
  mapping:=false \
  map_file:=$HOME/cpe631_ws/src/CPE631-Navigation-ROS2/maps/cafe_abs.yaml \
  enable_peds:=false
```

Fig. 2. Terminal command used to launch the static navigation test with pedestrians disabled.

```
bhagyath@bbr:~/cpe631_ws$ cd ~/cpe631_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
export RMW_FASTRTPS_USE_SHM=0

ros2 launch cpe631_ros2 cafe.launch.py \
  navigation:=true \
  mapping:=false \
  map_file:=$HOME/cpe631_ws/src/CPE631-Navigation-ROS2/maps/cafe_abs.yaml \
  params_file:=$HOME/cpe631_ws/src/CPE631-Navigation-ROS2/param/nav2_dynamic_conservative_navfn_dwb.yaml \
  enable_peds:=true
```

Fig. 3. Terminal command used to launch the dynamic navigation test with pedestrians enabled.

In the dynamic setup, pedestrians were enabled. This created a more challenging condition because moving humans could cross the robot's path. The dynamic setup was used for the main comparison between NavFn with MPPI and NavFn with DWB.

V. Implementation Details

This section explains the step-by-step implementation procedure followed in the project. The complete navigation workflow was divided into map construction, static navigation verification, dynamic navigation testing, planner-controller configuration, and metrics collection. This helped verify each part of the navigation stack before comparing the controller performance.

A. Teleoperation and Map Construction

The first step was to generate the cafe environment map. The TurtleBot3 Burger robot was launched in the Gazebo cafe simulation, and the robot was manually driven using teleoperation. During this process, the robot explored the indoor environment, including walls, tables, and open navigation

areas. The purpose of this step was to allow the mapping system to observe the environment and build an occupancy grid map.

After the robot explored the main regions of the cafe environment, the generated map was saved as a map file. This saved map was later used for localization and navigation testing. Using the same map for all test cases ensured that both controller configurations were evaluated under the same environmental conditions.

```
bhagyath@bbr:~/cpe631_ws$ cd ~/cpe631_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
export TURTLEBOT3_MODEL=burger
export RMW_FASTRTPS_USE_SHM=0

ros2 run turtlebot3_teleop teleop_keyboard
```

Fig. 4. Keyboard teleoperation launch command.

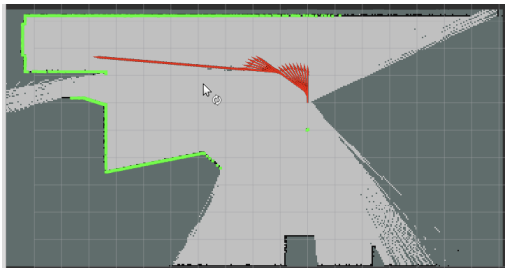

Fig. 5. RViz view during teleoperation-based map construction.

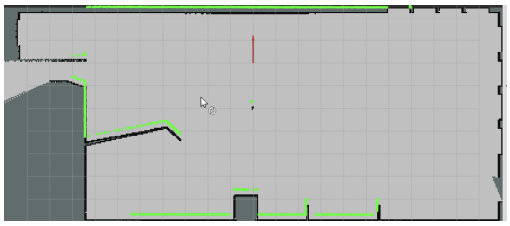

Fig. 6. Saved cafe occupancy grid map used for navigation.

```
bhagyath@bbr:~/cpe631_ws$ mkdir -p ~/cpe631_ws/maps

ros2 run nav2_map_server map_saver_cli \
  -f ~/cpe631_ws/maps/cafe
```

Fig. 7. Map saving command using Nav2 map saver.

B. Static Navigation Verification

After the map was created, static navigation was tested first. In this setup, pedestrians were disabled, and the robot navigated only around fixed obstacles. This stage was important because it verified that the map server, AMCL localization, global planner, local controller, and RViz goal interface were working correctly before adding moving pedestrians.

The robot was localized on the map using the 2D Pose Estimate tool in RViz. After localization, a goal was assigned using the 2D Goal Pose tool. The robot then planned a path and moved toward the goal while avoiding static obstacles. This test confirmed that the basic Nav2 navigation pipeline was working properly.

```
bhagyath@bbr:~/cpe631_ws$ cd ~/cpe631_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
export TURTLEBOT3_MODEL=burger
export RMW_FASTRTPS_USE_SHM=0

ros2 launch cpe631_ros2 cafe.launch.py \
  navigation:=true \
  mapping:=false \
  map_file:=$HOME/cpe631_ws/src/CPE631-Navigation-ROS2/maps/cafe_abs.yaml \
  enable_peds:=false
```

Fig. 8. Static navigation launch command with pedestrians disabled.

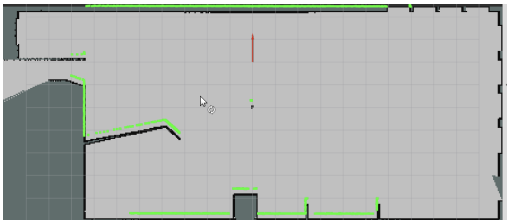

Fig. 9. Static navigation verification in RViz.

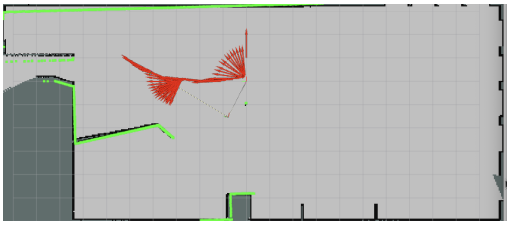

Fig. 10. RViz path and robot response during static navigation testing.

C. Dynamic Navigation Testing

After static navigation was verified, pedestrians were enabled in the cafe simulation. The dynamic setup introduced moving obstacles that could cross the robot path during navigation. This made the problem closer to a real indoor human environment.

In the dynamic test, the robot was again localized in RViz, and navigation goals were assigned using the 2D Goal Pose tool. The robot used laser scan data and the local costmap to detect nearby obstacles and pedestrians. The local costmap and collision monitoring concepts were used to support safe navigation around nearby obstacles and pedestrians [5], [6]. The dynamic navigation tests were used to compare how the two controller configurations responded to moving pedestrians.

```
bhagyath@bbr:~/cpe631_ws$ cd ~/cpe631_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
export RMW_FASTRTPS_USE_SHM=0

ros2 launch cpe631_ros2 cafe.launch.py \
  navigation:=true \
  mapping:=false \
  map_file:=$HOME/cpe631_ws/src/CPE631-Navigation-ROS2/maps/cafe_abs.yaml \
  params_file:=$HOME/cpe631_ws/src/CPE631-Navigation-ROS2/param/nav2_dynamic_conservative_navfn_dwb.yaml \
  enable_peds:=true
```

Fig. 11. Dynamic navigation launch command with pedestrians enabled.

Fig. 12. RViz view of the dynamic cafe navigation setup.

D. Planner and Controller Configuration

The global planner was kept the same for both experiments. NavFn was selected as the global planner because the main purpose of this project was to compare local controller behavior, not global planner behavior. Keeping the global planner fixed allowed a fair comparison between MPPI and DWB.

For the first configuration, NavFn was used with the MPPI controller. For the second configuration, NavFn was used with the DWB controller. The configuration files were modified to change only the controller plugin while keeping the same map, robot model, and pedestrian-enabled simulation environment.

The active planner and controller were verified using terminal commands before each test. This confirmed that the correct plugin was loaded during each experiment. The verification step was important because using the wrong configuration would make the controller comparison invalid.

E. Metrics Collection

A custom metrics logger was used during each navigation run. The logger recorded the robot path distance, total navigation time, minimum scan distance, and number of close encounters below a selected threshold. The timer was started when the robot began moving, and the logger was stopped after the robot reached the goal or stopped near the goal.

The collected metrics were used to compare the navigation performance of the two controller configurations. Navigation time was used to evaluate efficiency. Path distance was used to observe whether the robot followed a shorter or longer trajectory. Minimum scan distance and close encounters were used to evaluate how close the robot came to pedestrians or obstacles during navigation.

```
bhagyath@bbr:~/cpe631_ws$ cd ~/cpe631_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
export RMW_FASTRTPS_USE_SHM=0

python3 ~/cpe631_ws/nav_metrics_logger.py --run navfn_mppi_test1
```

Fig. 13. Terminal command used to start the custom ROS 2 navigation metrics logger.

F. Experimental Procedure

For each controller configuration, the same general testing procedure was followed. First, the navigation stack was launched with the selected parameter file. Then, the robot pose was initialized in RViz. After that, the metrics logger was started, and a navigation goal was assigned using RViz. The robot motion was observed in both Gazebo and RViz until the run ended.

The same process was repeated for NavFn with MPPI and NavFn with DWB. This ensured that the comparison was based on similar simulation conditions. The final results were analyzed using both numerical metrics and qualitative observations from the robot behavior.

VI. Navigation Parameter Tuning

After verifying that the robot could launch correctly in the cafe environment, the navigation parameters were reviewed and adjusted for the two dynamic navigation configurations used in this project. The global planner was kept as NavFn in both cases so that the effect of the local controller could be compared more clearly. The two local controllers used were MPPI and DWB.

A. NavFn with MPPI Parameter Settings

For the first dynamic navigation configuration, the NavFn planner was used with the MPPI controller. NavFn generated the global path using the saved cafe map, while MPPI generated local velocity commands for following the path and reacting to pedestrians and obstacles [3].

The main MPPI parameters considered in this setup were the robot velocity limits, angular velocity limits, acceleration limits, prediction horizon, trajectory sampling, and obstacle critic settings. These parameters directly affected how fast the robot moved, how smoothly it followed the path, and how strongly it reacted to nearby obstacles.

- `vx_max`: controlled the maximum forward speed of the robot.
- `wz_max`: controlled the maximum angular speed during turning.
- `batch_size`: controlled the number of sampled trajectories evaluated by MPPI.
- `time_steps` and `model_dt`: controlled the MPPI prediction horizon.
- `PathAlignCritic` and `PathFollowCritic`: helped the robot stay close to the planned global path.
- `GoalCritic`: helped the robot move toward the assigned goal.
- `CostCritic`: helped the robot react to nearby obstacles in the local costmap.

The MPPI configuration used the following important values:

```
vx_max: 0.70
wz_max: 1.50
batch_size: 2000
time_steps: 56
model_dt: 0.05
PathAlignCritic.cost_weight: 14.0
PathFollowCritic.cost_weight: 5.0
GoalCritic.cost_weight: 5.0
```

```
CostCritic.cost_weight: 3.81
xy_goal_tolerance: 0.30
yaw_goal_tolerance: 0.65
```

The goal checker values were important because they determined when the robot considered the navigation task complete. A slightly relaxed yaw tolerance was used so that the robot could complete the goal without excessive rotation near pedestrians or obstacles. These settings helped MPPI generate smoother local motion while still allowing the robot to reach the goal in the dynamic cafe environment.

B. NavFn with DWB Parameter Settings

For the second dynamic navigation configuration, the NavFn planner was kept as the global planner, and the local controller was changed to DWB. NavFn generated the global path on the saved cafe map, while DWB worked as the local controller by sampling possible velocity commands and scoring each candidate trajectory using a set of critics [2], [4]. The trajectory with the best score was selected as the local motion command.

The main DWB parameters considered in this setup were the velocity limits, acceleration limits, trajectory sampling values, simulation time, and critic weights. These values controlled how the robot selected local paths around obstacles, followed the global path, and approached the assigned goal [4].

- `max_vel_x`: controlled the maximum forward speed.
- `max_vel_theta`: controlled the maximum rotational speed.
- `vx_samples` and `vtheta_samples`: controlled how many velocity samples were evaluated.
- `sim_time`: controlled how far ahead each sampled trajectory was simulated.
- `BaseObstacle.scale`: affected obstacle avoidance behavior.
- `PathAlign.scale` and `PathDist.scale`: helped the robot stay close to the global path.
- `GoalAlign.scale` and `GoalDist.scale`: helped the robot move toward the assigned goal.
- `RotateToGoal.scale`: helped the robot align near the goal pose.

The DWB configuration used the following important values:

```
max_vel_x: 0.35
max_vel_theta: 1.0
vx_samples: 20
vtheta_samples: 20
sim_time: 1.7
BaseObstacle.scale: 0.08
PathAlign.scale: 32.0
GoalAlign.scale: 24.0
PathDist.scale: 32.0
GoalDist.scale: 24.0
RotateToGoal.scale: 32.0
```

The same goal checker settings were used for both MPPI and DWB so that the comparison remained consistent.

```
xy_goal_tolerance: 0.30
yaw_goal_tolerance: 0.65
```

By keeping the global planner the same and changing only the local controller, the project was able to compare how MPPI and DWB behaved in the same dynamic cafe navigation environment.

VII. Results and Performance Analysis

This section presents the results obtained from the dynamic navigation experiments. Two test cases were performed for each controller configuration. Test 1 used the normal dynamic cafe environment with moving pedestrians. Test 2 used the same environment with additional obstacles such as cylinders and cubes added manually to increase navigation difficulty.

The same global planner, map, robot model, and pedestrian-enabled environment were used for both configurations. The main difference between the two configurations was the local controller. The first configuration used NavFn with MPPI, while the second configuration used NavFn with DWB.

A. NavFn Planner with MPPI Controller Results

The first dynamic navigation configuration tested in this project used the NavFn global planner with the MPPI local controller. In this setup, NavFn generated the global path using the saved cafe map, while the MPPI controller generated the local velocity commands needed to follow the path and react to nearby obstacles and moving pedestrians. Two tests were performed for this configuration: one in the normal dynamic cafe environment and one with additional manually placed obstacles.

1) Test 1: Normal Dynamic Environment: In Test 1, the robot was tested in the dynamic cafe environment with moving pedestrians enabled. No additional obstacles were manually added to the environment. The purpose of this test was to evaluate the basic dynamic navigation performance of the NavFn and MPPI configuration in the original cafe scenario.

The robot was first localized on the saved cafe map, and then a goal pose was assigned in RViz. After the goal was provided, the NavFn planner generated a global path toward the target location. The MPPI controller followed this path while continuously adjusting the robot motion based on local obstacle information from the costmap. During the test, the robot was able to move through the dynamic environment and navigate toward the assigned goal while responding to pedestrian movement.

The recorded metrics showed that the robot completed the navigation task in 251.41 seconds with a total path distance of 9.91 m. The minimum scan distance recorded during the run was 0.13 m, and the robot had 2 encounters below the selected 1 m threshold. These results show that the robot was able to reach the goal in the normal dynamic environment, but it moved close to pedestrians or obstacles during the run.

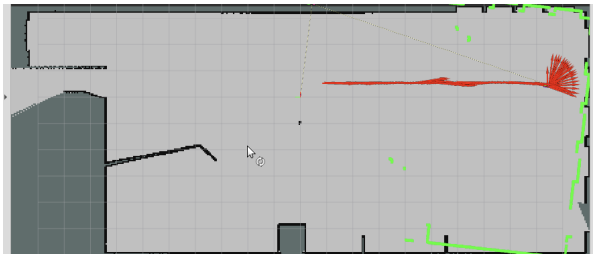

Fig. 14. RViz result for NavFn with MPPI Test 1

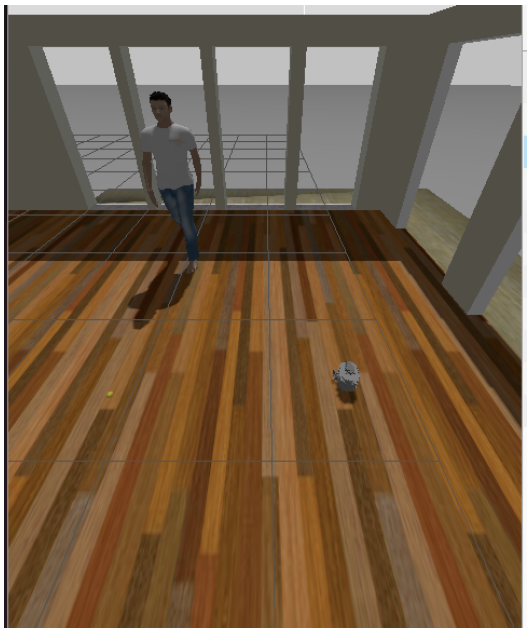

Fig. 15. Gazebo final pose of the TurtleBot3.

```
Total navigation time          : 251.41 sec
Robot path distance            : 9.91 m
Encounters below 1.0 m         : 2
Minimum scan distance          : 0.13 m
CSV saved at                   : /tmp/nav_metrics.csv
```

Fig. 16. Navigation metrics recorded for NavFn with MPPI Test 1.

2) Test 2: Dynamic Environment with Additional Obstacles: In Test 2, the same NavFn and MPPI configuration was tested again, but additional obstacles were manually placed in the environment. Cylinders and cubes were added to reduce the available free space and make the navigation task more challenging. This test was used to observe how the robot behaved when both static obstacles and moving pedestrians were present.

The robot was again localized on the saved cafe map, and a new goal pose was assigned in RViz. The NavFn planner generated a global path to the goal, and the MPPI controller attempted to follow the path while avoiding the added obstacles and moving pedestrians. Compared with Test 1, this case was more difficult because the robot had to adjust its local motion more frequently due to the extra obstacles in the environment.

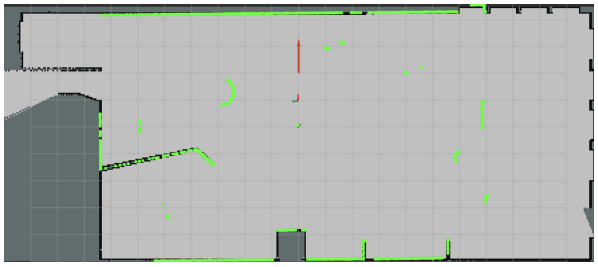

Fig. 17. RViz result for NavFn with MPPI Test 2.

The recorded metrics showed that the robot took 711.04 seconds to complete the navigation task, with a total path distance of 9.29 m. The minimum scan distance was 0.24 m, and the robot had 3 encounters below the selected 1 m threshold. Compared with Test 1, the navigation time increased significantly because the robot spent more time adjusting its path around the added obstacles and pedestrians. However, the slightly larger minimum scan distance indicates that the robot maintained a somewhat safer clearance during this more difficult test.

Overall, the NavFn with MPPI configuration was able to complete both dynamic navigation tests. The results suggest that MPPI can react to dynamic obstacles and added environmental constraints, but the navigation time increases when the environment becomes more cluttered. These results are later compared with the NavFn with DWB configuration to evaluate which local controller performs better under similar dynamic navigation conditions.

Fig. 18. Gazebo view for NavFn with MPPI Test 2.

B. NavFn Planner with DWB Controller Results

The second dynamic navigation configuration tested in this project used the NavFn global planner with the DWB local controller. The global planner was kept the same as the previous configuration so that the effect of changing only the local controller could be observed. In this setup, NavFn generated the global path on the saved cafe map, while DWB selected local velocity commands by evaluating possible trajectories and scoring them based on path following, goal progress, obstacle clearance, and heading behavior. Two tests

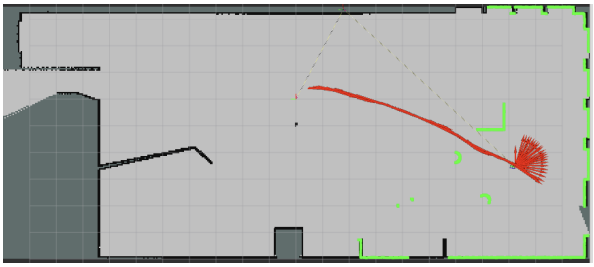

Fig. 19. RViz result for NavFn with MPPI Test 2.

```
Total navigation time        : 711.04 sec
Robot path distance          : 9.29 m
Encounters below 1.0 m       : 3
Minimum scan distance        : 0.24 m
CSV saved at                 : /tmp/nav_metrics.csv
```

Fig. 20. Metrics for NavFn with MPPI Test 2.

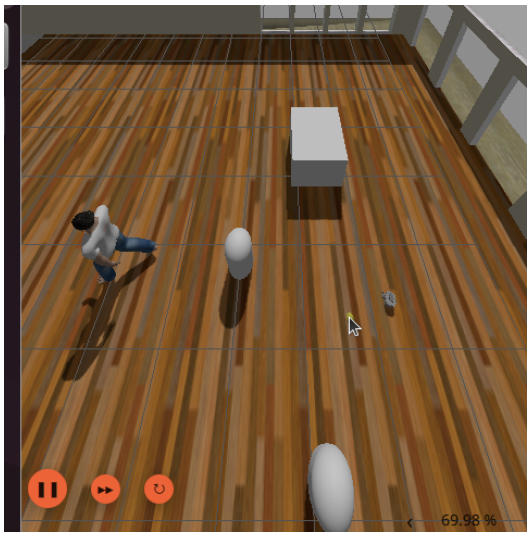

Fig. 21. Robot final goal pose.

were performed for this configuration: one in the normal dynamic cafe environment and one with additional obstacles added manually.

1) Test 1: Normal Dynamic Environment: In Test 1, the NavFn and DWB configuration was tested in the original dynamic cafe environment with pedestrians enabled. The robot was localized on the saved cafe map and then commanded to move toward a selected goal pose using RViz. This test was used to evaluate how the DWB controller performed in the pedestrian-enabled environment without adding extra obstacles.

During this test, NavFn generated the global path, and DWB selected local velocity commands based on trajectory scoring. The robot followed the planned path while reacting to obstacles detected in the local costmap. The controller was able to guide the robot through the environment and complete the assigned navigation task.

The recorded metrics showed that the robot completed the navigation task in 240.57 seconds with a total path distance of 4.28 m. The minimum scan distance recorded during the run was 0.91 m, and the robot had 3 close encounters below the selected 1 m threshold. These results show that the NavFn with DWB configuration was able to reach the goal in the normal dynamic environment while maintaining a reasonable distance from pedestrians and obstacles.

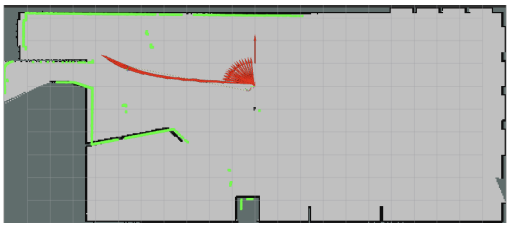

Fig. 22. RViz result for NavFn with DWB Test 1.

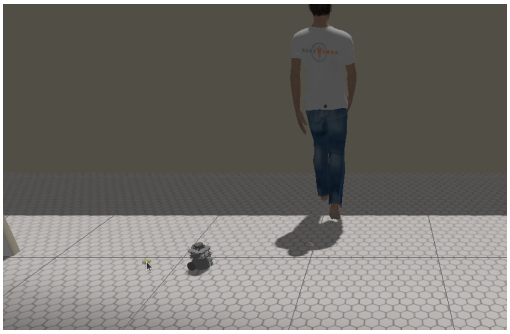

Fig. 23. Gazebo view for NavFn with DWB Test 1.

```
FINAL NAVIGATION METRICS
Run name                     : navfn_dwb_test1
Total navigation time        : 240.57 sec
Robot path distance          : 4.28 m
Encounters below 1.0 m       : 3
Minimum scan distance        : 0.91 m
CSV saved at                 : /tmp/navfn_dwb_test1_nav_metrics.csv
```

Fig. 24. Metrics for NavFn with DWB Test 1.

In Test 2, the same NavFn and DWB configuration was tested again, but additional obstacles were manually placed in the cafe environment. Cylinders and cubes were added to reduce the available free space and increase the navigation difficulty. This test was used to observe how the DWB controller behaved when the robot had to avoid both manually added obstacles and moving pedestrians.

The robot was again localized on the saved cafe map, and a new goal pose was assigned in RViz. NavFn generated the global route toward the target location, while DWB generated local velocity commands to follow the path and avoid nearby obstacles. Since the environment was more crowded in this test, the robot had less free space for navigation and needed to make more local adjustments compared with the normal dynamic environment.

The recorded metrics showed that the robot completed the navigation task in 393.73 seconds with a total path distance

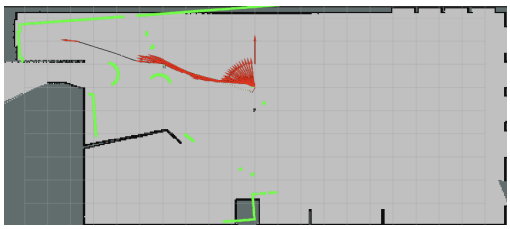

Fig. 25. Robot navigating near pedestrians in RViz during DWB Test 2.

of 5.76 m. The minimum scan distance recorded during this test was 0.32 m, and the robot had 2 close encounters below the selected 1 m threshold. Compared with the normal dynamic test, the navigation time and path distance increased because the robot had to move through a more constrained environment. However, the robot was still able to complete the task and reach the assigned goal.

Overall, the NavFn with DWB configuration successfully completed both dynamic navigation tests. The results indicate that DWB was able to follow the global path and avoid obstacles in both the normal cafe environment and the more difficult environment with additional obstacles. The increase in navigation time and path distance in Test 2 shows the effect of the added obstacles on local trajectory selection and robot motion.

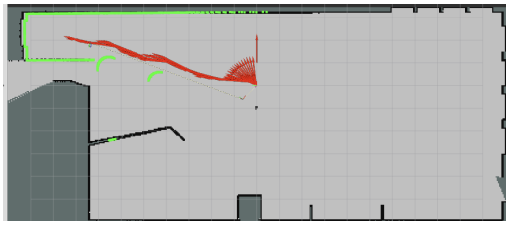

Fig. 26. RViz path response during DWB Test 2.

Fig. 27. Gazebo view of the robot navigating around added obstacles during DWB Test 2.

```
FINAL NAVIGATION METRICS
Run name                : navfn_dwb_test2
Total navigation time   : 393.73 sec
Robot path distance     : 5.76 m
Encounters below 1.0 m  : 2
Minimum scan distance   : 0.32 m
CSV saved at            : /tmp/navfn_dwb_test2_nav_metrics.csv
```

Fig. 28. Navigation metrics recorded for NavFn with DWB Test 2.

C. Result Summary

The four dynamic navigation tests were evaluated using both recorded metrics and visual observations from RViz and Gazebo. The main metrics considered were total navigation time, total path distance, minimum scan distance, and the number of close encounters below the selected 1 m threshold. These values helped compare how each controller behaved in the normal dynamic cafe environment and in the more difficult environment with additional obstacles.

TABLE I
PERFORMANCE COMPARISON OF NAVFN WITH MPPI AND NAVFN WITH DWB.

Metric	MPPI T1	MPPI T2	DWB T1	DWB T2
Time (sec)	251.41	711.04	240.57	393.73
Distance (m)	9.91	9.29	4.28	5.76
Min scan (m)	0.13	0.24	0.91	0.32
Close encounters	2	3	3	2

For the NavFn with MPPI configuration, the robot completed both tests, but the second test required significantly more time because the added obstacles reduced the available free space and forced the robot to make more local adjustments. The path distance stayed high in both tests, showing that the robot followed a longer and more cautious route. However, the MPPI controller was still able to continue moving toward the goal while reacting to pedestrians and obstacles.

For the NavFn with DWB configuration, the robot also completed both tests. In the normal dynamic environment, the robot reached the goal with a shorter path distance compared with the MPPI test. In the additional obstacle case, the navigation time and path distance increased, which shows that the controller had to adjust its trajectory more frequently in the constrained environment. The DWB controller maintained goal-directed motion and completed the task even when the environment became more crowded.

Overall, both configurations were able to complete the dynamic navigation tasks. Since the global planner was kept the same for all tests, the observed differences mainly came from the local controller behavior. The MPPI controller showed smoother local adjustment in dynamic conditions, while the DWB controller provided a more direct trajectory in some cases but required careful tuning to handle crowded regions. These results show that local controller selection has a strong effect on navigation time, path length, and obstacle-avoidance behavior in the simulated cafe environment.

VIII. DISCUSSION

The results from this project show that the choice of local controller has a clear effect on robot navigation behav-

ior in a dynamic indoor environment. Since the same cafe map, TurtleBot3 Burger model, NavFn global planner, and pedestrian-enabled simulation were used for both controller configurations, the main difference between the tests came from the local controller. This made the comparison between MPPI and DWB more consistent.

The MPPI controller was able to guide the robot through the dynamic cafe environment while reacting to pedestrians and nearby obstacles. In the normal dynamic test, the robot completed the navigation task while maintaining motion toward the goal. In the additional-obstacle test, the navigation time increased because the robot had to adjust its motion more frequently. This behavior was expected because the extra cylinders and cubes reduced the free navigation space and made the local planning problem more difficult.

The DWB controller also completed both navigation tests. In the normal dynamic environment, DWB produced a direct path toward the goal and completed the test with a shorter path distance. In the additional-obstacle environment, the robot still reached the goal, but the navigation became more challenging because the controller had to select feasible velocity commands while avoiding both static and moving obstacles. This shows that DWB can work well in structured environments, but its behavior depends strongly on tuning parameters such as velocity limits, trajectory scoring critics, and obstacle-cost weighting.

One important lesson from the project was that a working navigation result depends on more than only selecting a planner and controller. Several parts of the Nav2 stack had to work correctly together, including map loading, AMCL localization, costmap updates, planner activation, controller activation, and goal execution through the BT navigator. During testing, terminal verification was important to confirm that the correct planner and controller plugins were actually loaded before collecting results. Without this verification, it would be difficult to make a fair comparison between the two configurations.

Another challenge was working with dynamic pedestrians. Pedestrians made the navigation problem more realistic because they created moving obstacles that could cross the robot path. However, they also made the results less perfectly repeatable because pedestrian positions changed during each run. For this reason, both numerical metrics and visual observations were used. The metrics helped compare navigation time, distance, and minimum scan distance, while RViz and Gazebo helped explain the robot behavior during each test.

Regarding social navigation behavior, both controller configurations treated moving pedestrians as dynamic obstacles detected through laser scan data and the local costmap. The robot was able to react to pedestrians and avoid collisions, but the motion was not fully socially aware because no dedicated human-aware or social-force model was implemented. MPPI showed continuous local trajectory adjustment in dynamic scenes, while DWB maintained more direct goal-directed motion through trajectory scoring. However, the close-encounter and minimum scan distance results show that both controllers sometimes moved close to pedestrians, so future improvement should include explicit social navigation constraints such as personal-space cost functions or human-aware path planning.

Overall, this project provided practical experience with ROS 2 Nav2 navigation in both static and dynamic environments. The work helped show how global planning, local control, localization, costmaps, and simulation tools interact in a complete navigation pipeline. The comparison between NavFn with MPPI and NavFn with DWB showed that both controllers can complete the navigation task, but their behavior differs in path smoothness, response to obstacles, and sensitivity to tuning.

IX. Conclusion

This project implemented and evaluated autonomous navigation for a TurtleBot3 Burger robot in a simulated cafe environment using ROS 2 Jazzy, Gazebo, RViz, and the Nav2 navigation stack. The work included map construction through teleoperation, loading the saved cafe map, localizing the robot using AMCL, and testing navigation in both static and dynamic environments with moving pedestrians.

The main comparison focused on two navigation configurations: NavFn with MPPI and NavFn with DWB. The NavFn planner was kept the same in both configurations so that the effect of the local controller could be compared more clearly. Both controller configurations were tested in a normal dynamic cafe environment and in a more difficult environment with additional manually placed obstacles. For each test, navigation time, robot path distance, minimum scan distance, and close encounters below the selected threshold were recorded.

The results showed that both MPPI and DWB were able to navigate the robot toward the assigned goal in the dynamic environment. The MPPI controller showed effective local motion adjustment around pedestrians and obstacles, while the DWB controller provided a trajectory-scoring-based approach that also completed the navigation task. The additional-obstacle tests were more challenging and produced higher navigation difficulty because the robot had less free space and had to react more frequently to local obstacles.

Overall, the project demonstrated the importance of correctly configuring and verifying the Nav2 pipeline before collecting results. Map loading, AMCL localization, planner activation, controller activation, costmap behavior, and terminal plugin verification were all necessary steps for reliable testing. The project also showed that local controller selection and tuning have a strong effect on navigation performance in human-populated indoor environments.

Future work can include testing more planner and controller combinations, tuning the controller parameters more systematically, and performing repeated trials to obtain more statistically consistent results. Additional work can also include evaluating navigation performance with different pedestrian densities and using more advanced behavior strategies for human-aware navigation.

REFERENCES

[1] SIT Robotics and Automation Laboratory, "CPE631-Navigation-ROS2," GitHub repository. [Online]. Available: https://github.com/SIT-Robotics-and-Automation-Laboratory/CPE631-Navigation-ROS2

[2] Open Robotics, "nav2_navfn_planner," ROS Index. [Online]. Available: https://index.ros.org/p/nav2_navfn_planner/#humble-deps

[3] Open Robotics, "nav2_mppi_controller," ROS 2 Documentation. [Online]. Available: https://docs.ros.org/en/iron/p/nav2_mppi_controller/

[4] Open Robotics, "dwb_local_planner," ROS Wiki. [Online]. Available: https://wiki.ros.org/dwb_local_planner

[5] Open Robotics, "nav2_costmap_2d," ROS Index. [Online]. Available: https://index.ros.org/p/nav2_costmap_2d/

[6] Open Robotics, "nav2_collision_monitor," ROS 2 Documentation. [Online]. Available: https://docs.ros.org/en/humble/p/nav2_collision_monitor/

[7] Open Robotics, "ROS 2 Jazzy Documentation." [Online]. Available: https://docs.ros.org/en/jazzy/index.html

[8] Open Robotics, "Gazebo Simulation with ROS 2," ROS 2 Documentation. [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Advanced/Simulators/Gazebo/Gazebo.html